1. Hospital Doctor Scheduling Using Backtracking Search

Problem Statement

A hospital needs to assign three doctors (D1, D2, and D3) to three shifts: Morning, Afternoon, and Night. The assignment must satisfy all the given constraints. Backtracking Search is used to systematically explore possible assignments and reject any assignment that violates a constraint.

Problem Formulation

Variables: D1, D2, D3

Domains: {Morning, Afternoon, Night}

Constraints:

1. D1 cannot work the Night shift.
2. D2 must work before D3, following the order Morning < Afternoon < Night.
3. Only one doctor can be assigned to each shift.
4. D3 cannot work the Morning shift.
5. Every doctor must be assigned exactly one shift.

Method Used: Backtracking Search

Backtracking Search assigns a value to one variable at a time. After each assignment, the constraints are checked. If an assignment violates a constraint, the algorithm rejects it and tries another value. If no valid value remains, it backtracks to the previous variable and changes its assignment.

Step-by-Step Solution

For clarity, the shifts are represented as:

Morning = 1, Afternoon = 2, Night = 3.

The constraint D2 < D3 means that D2 must be assigned an earlier shift than D3.

Step 1: Assign D1

Initially, the domains of all three doctors are:

D1 = {Morning, Afternoon}
D2 = {Morning, Afternoon, Night}
D3 = {Afternoon, Night}

D1 cannot work Night, so that value is removed from its domain.

Try assigning D1 = Morning.

Constraint check: D1 is not assigned Night, so this assignment is valid so far.

Step 2: Assign D2

Try D2 = Afternoon.

Constraint check: D1 and D2 occupy different shifts, so the assignment satisfies the one-doctor-per-shift constraint.

Step 3: Assign D3

The remaining shift is Night, so assign D3 = Night.

Constraint checks:

- D1 is assigned Morning, not Night.
- D3 is assigned Night, not Morning.
- All three doctors have different shifts.
- D2 is assigned Afternoon, which comes before D3's Night shift.
- Every doctor has exactly one shift.

All constraints are satisfied.

Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Backtracking Search Pseudocode

```
BACKTRACK(assignment):

    IF all doctors are assigned:
        RETURN assignment

    Select an unassigned doctor D
```

```
  FOR each shift in D's domain:

    IF shift satisfies all constraints:
      Assign D to shift

      result = BACKTRACK(assignment)

      IF result is not failure:
        RETURN result

      Remove D's assignment

  RETURN failure
```

Result and Interpretation

The final schedule is D1–Morning, D2–Afternoon, and D3–Night. This assignment satisfies all five constraints, ensuring that every doctor receives exactly one shift, no two doctors share a shift, D1 does not work Night, D3 does not work Morning, and D2 works before D3.

Backtracking Search is appropriate because it systematically assigns shifts, checks constraints at each step, and rejects invalid assignments before continuing.

2. Robot Navigation in a 5×5 Grid Using BFS Search Algorithm

Problem Statement

A robot operates in a 5×5 grid and must move from the Start (S) to the Goal (G). It must avoid obstacles and find the shortest possible path.

The given constraints are:

1. The robot can move Up, Down, Left, or Right.
2. Each movement has a cost of 1.
3. The robot cannot pass through obstacles.
4. Manhattan Distance must be used as the heuristic.

Note: The question paper does not specify the exact positions of the start, goal, or obstacles. Therefore, the following grid is an assumed example to demonstrate the search process.

Step 1: Represent the Grid

- S – Starting position
- G – Goal position
- Solid Box - Obstacle
- Empty cell – Available path

S			**#**	
	#		#	
	#			
			#	
#	#			**G**

The robot starts at position (1,1) and must reach (5,5). Coordinates are represented as (row, column).

Step 2: Calculate Manhattan Distance

Manhattan Distance estimates the distance between the current position and the goal without considering diagonal movement.

The formula is:

$h(n) = |x_1 - x_2| + |y_1 - y_2|$

Where:

- (x_1, y_1) is the current position.
- (x_2, y_2) is the goal position.

For the starting position (1,1) and goal position (5,5):

$h(S) = |1 - 5| + |1 - 5|$

$h(S) = 4 + 4 =$ **8**

Therefore, the Manhattan Distance from the start to the goal is 8.

Step 3: Apply Breadth-First Search (BFS)

BFS explores nodes level by level using a FIFO (First In, First Out) queue. It checks neighbouring cells in the order Up, Down, Left, Right.

A cell is added to the queue only if it is inside the grid, is not an obstacle, and has not already been visited.

Step	Node Expanded	Valid Neighbours Added	Queue After Expansion
1	(1,1)	(2,1), (1,2)	[(2,1), (1,2)]
2	(2,1)	(3,1)	[(1,2), (3,1)]
3	(1,2)	(1,3)	[(3,1), (1,3)]
4	(3,1)	(4,1)	[(1,3), (4,1)]
5	(1,3)	(2,3)	[(4,1), (2,3)]
6	(4,1)	(4,2)	[(2,3), (4,2)]
7	(2,3)	(3,3)	[(4,2), (3,3)]
8	(4,2)	(4,3)	[(3,3), (4,3)]
9	(3,3)	(3,4)	[(4,3), (3,4)]
10	(4,3)	(5,3)	[(3,4), (5,3)]
11	(3,4)	(3,5)	[(5,3), (3,5)]
12	(5,3)	(5,4)	[(3,5), (5,4)]

13	(3,5)	(4,5)	[(5,4), (4,5)]
14	(5,4)	(5,5)	[(4,5), (5,5)]
15	(4,5)	None	[(5,5)]
16	(5,5)	Goal reached	[]

Step 4: Determine the Optimal Path

The path from the starting position to the goal is:

S → (2,1) → (3,1) → (4,1) → (4,2) → (4,3) → (5,3) → (5,4) → G

The robot reaches the goal after making 8 movements.

Step 5: Calculate the Total Path Cost

Each movement has a cost of 1.

Path Cost = Number of Movements × Cost per Movement

Path Cost = 8 × 1

Total Path Cost = 8

The path uses only valid cells and avoids all obstacles. Eight is also the Manhattan Distance between the start and goal, so no shorter path is possible in this grid.

Step 6: BFS Algorithm Pseudocode

```
BFS(start, goal):

    Create an empty FIFO queue
    Add start to the queue
    Mark start as visited
    Store the parent of each node

    While the queue is not empty:

        Remove the front node

        If the node is goal:
            Return the path and its cost

        For each neighbour in Up, Down, Left, Right:
```

```
        If neighbour is inside the grid
           and is not an obstacle
           and is not visited:

            Mark neighbour as visited
            Store its parent
            Add neighbour to the queue

    Return "No path exists"
```

Note on BFS, Manhattan Distance, and Priority Queue

The question asks for BFS, Manhattan Distance, and priority queue updates. However, standard BFS uses a FIFO queue and does not prioritize nodes using a heuristic. Manhattan Distance is commonly used in heuristic-based algorithms such as A* Search and Greedy Best-First Search.

Therefore, the table above shows the queue updates for BFS, while Manhattan Distance is calculated separately as requested. A priority queue would be appropriate if the intended algorithm were A* or Greedy Best-First Search.

Result

The robot reaches the goal while avoiding obstacles. For the assumed grid, the optimal path consists of **8 movements**, with a total cost of **8**. BFS is suitable because every movement has the same cost and it finds the shortest path when a solution exists.

3. Autonomous Rescue Robot Using AI Search Strategies

Problem Statement

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The building is represented as a grid of rooms, and the robot must travel from the starting position S to the survivor's location G.

The environment is partially observable because the robot can observe only adjacent cells. Some cells contain obstacles that cannot be traversed. Each movement has a cost of 1, while entering a risky zone incurs an additional cost of 2. The robot must minimize the total travel cost, navigate safely, and update its knowledge of the environment dynamically.

a) Analysis of Constraints and Their Effect on Decision-Making

1. Movement Restrictions

The robot can move only Up, Down, Left, or Right. Diagonal movements are not allowed.

Effect on decision-making: The robot must select a valid neighbouring cell at every step and cannot move diagonally to shorten its route.

2. Obstacles

Some cells are blocked and cannot be traversed.

Effect on decision-making: The robot must identify blocked cells using its sensors and avoid entering them. If a planned path becomes blocked, it must explore an alternative route.

3. Limited Sensing Capability

The robot can observe only adjacent cells and cannot access complete information about the building in advance.

Effect on decision-making: The robot must make decisions using the information currently available. It must explore the environment gradually and update its internal representation whenever new information is obtained.

4. Movement Cost

Every movement has a cost of 1.

Effect on decision-making: The robot should avoid unnecessary movements and prefer routes with lower cumulative costs.

5. Risky Zones

Entering a risky zone adds an additional cost of 2 to the normal movement cost of 1.

Therefore:

Cost of entering a normal cell = 1

Cost of entering a risky cell = 1 + 2 = 3

Effect on decision-making: The robot should consider both distance and risk. A longer route through safe cells may be preferable to a shorter route through risky areas if its total cost is lower.

6. Dynamic Knowledge Updates

The robot must update its knowledge as it discovers new obstacles, safe cells, and risky zones.

Effect on decision-making: The robot must revise its route when new information becomes available rather than relying entirely on an initial plan.

b) Solution Approach Using Uniform Cost Search and Online Search

Selected Strategy: Uniform Cost Search (UCS) with Online Exploration

Uniform Cost Search is suitable for finding a minimum-cost path because different movements can have different costs. Normal movements cost 1, whereas entering a risky zone costs 3.

The robot can use UCS to compare the cumulative costs of discovered routes. Since the environment is partially observable, it must combine this strategy with online exploration to discover unknown cells and update its map.

Step 1: Initialize the Robot

The robot begins at position S with a cumulative cost of 0.

It initializes its known map using the information available from its sensors.

Step 2: Observe Adjacent Cells

The robot senses the cells immediately above, below, to the left, and to the right of its current position.

Each observed cell is classified as:

- Safe and traversable
- Risky but traversable
- Blocked and inaccessible

- Unknown until observed

Step 3: Calculate Movement Costs

The cost of entering a neighbouring cell is calculated as follows:

Cell Type	Movement Cost
Normal cell	1
Risky cell	3
Blocked cell	Not allowed

The cumulative cost is calculated using:

New cumulative cost = Current cumulative cost + Movement cost

Step 4: Select the Lowest-Cost Route

UCS maintains a priority queue ordered by cumulative path cost.

The robot selects the reachable, discovered position with the lowest cumulative cost for further exploration. Blocked cells are excluded.

If several positions have equal costs, a consistent tie-breaking rule can be used.

Step 5: Update the Map

After moving to a new position, the robot observes its surroundings and updates its internal map.

If a previously unknown obstacle or risky zone is discovered, the robot updates the corresponding cell information and recalculates the available routes.

Step 6: Continue Until the Survivor Is Reached

The robot continues exploring, updating its map, and selecting low-cost routes until it reaches G.

Once the survivor's location is reached, the robot completes the navigation phase and can initiate the rescue procedure.

Step 7: Calculate the Total Path Cost

The total cost is the sum of all movement costs along the selected path.

Total Path Cost = Number of Normal Moves + (3 × Number of Risky Cells Entered)

This formula assumes that each risky cell contributes a total entry cost of 3, including the normal movement cost.

The robot should select the least-cost feasible route based on the information discovered during exploration.

Pseudocode

```
ONLINE_RESCUE_ROBOT(S, G):

    Initialize known_map
    Initialize priority_queue
    Set cost(S) = 0

    Insert S into priority_queue
    with priority 0

    WHILE G has not been reached:

        Observe adjacent cells
        Update known_map

        Remove the discovered position
        with the lowest cumulative cost

        FOR each adjacent cell:

            IF cell is blocked:
                Skip the cell

            ELSE:
                IF cell is risky:
                    movement_cost = 3
                ELSE:
                    movement_cost = 1

                new_cost = current_cost
                        + movement_cost

                IF new_cost improves the
                  known cost of the cell:

                    Update the cell cost
```

Store its parent
Insert or update the
cell in priority_queue

Select the next reachable position
with the lowest cumulative cost

Move to the selected position
Observe the new surroundings
Update known_map

Reconstruct the path from S to G
Return the path and total cost

c) Application of AI Concepts

1. Intelligent Agents

An intelligent agent perceives its environment through sensors and performs actions using actuators to achieve its objectives.

In this scenario:

- **Sensors:** Detect nearby obstacles, safe paths, and risky zones.
- **Percepts:** Information about adjacent cells and their conditions.
- **Agent program:** Maintains the map, compares route costs, and selects actions.
- **Actuators:** Move the robot in the selected direction.
- **Goal:** Reach the survivor while minimizing travel cost and maintaining safe navigation.

The robot acts as an autonomous intelligent agent because it observes its environment, processes information, and chooses actions based on its objectives.

2. Rationality

A rational agent selects actions that maximize its expected performance according to the available information.

The rescue robot's performance criteria include:

- Reaching the survivor successfully
- Minimizing total movement cost
- Avoiding blocked cells
- Reducing exposure to risky zones
- Updating its decisions when new information is discovered

For example, suppose one route contains four normal moves and another contains two normal moves and one risky-cell entry.

Route A cost = 4 × 1 = 4

Route B cost = (2 × 1) + (1 × 3) = 5

The robot should prefer Route A if both routes are feasible and all other relevant conditions are equal.

This demonstrates rational decision-making based on cumulative cost rather than simply choosing the route with fewer movements.

3. Environment Type

The rescue robot operates in an environment with the following characteristics:

Environment Property	Classification	Explanation
Observability	Partially observable	The robot can observe only adjacent cells.
Determinism	Partially predictable	Movement costs are defined, but unknown obstacles and hazards create uncertainty about the environment.
Sequential/Episodic	Sequential	Each movement affects subsequent positions and available routes.
Static/Dynamic	Dynamic from the agent's perspective	The robot's knowledge changes as it explores and receives new observations.
Discrete/Continuous	Discrete	The building is represented as a grid of individual cells.
Single-agent/Multi-agent	Single-agent	The robot is the decision-making agent in the stated problem.
Known/Unknown	Partially unknown	The robot does not initially know the conditions of every cell.

These characteristics explain why the robot must make decisions using incomplete information and revise its plans as it explores.

d) Justification for the Selected Approach

Uniform Cost Search combined with online exploration is appropriate for the rescue robot for the following reasons:

1. Handles Different Movement Costs

Unlike BFS, which is optimal when all movement costs are equal, UCS accounts for different costs associated with normal and risky cells.

2. Finds Minimum-Cost Paths

When applied to a known graph with non-negative movement costs, UCS finds a minimum-cost path to the goal. This helps the robot compare safe and risky routes systematically.

3. Supports Exploration with Updated Knowledge

The robot can observe adjacent cells, update its map, and replan when new obstacles or hazards are discovered. This is necessary because the environment is partially observable.

4. Avoids Blocked Cells

The search process excludes cells that cannot be traversed, preventing the robot from selecting an invalid route.

5. Supports Rational Decisions

By comparing cumulative costs, the robot can select a feasible route that balances travel distance and risk.

Limitation

UCS by itself assumes that the graph being searched is known. Therefore, in this scenario, it must be combined with an online exploration mechanism that discovers the environment and triggers replanning. Also, minimizing the assigned cost does not guarantee physical safety unless the risk costs adequately represent the hazards and safety requirements.

Result

The autonomous rescue robot can use Uniform Cost Search with online exploration to navigate a partially observable disaster environment. By observing adjacent cells, avoiding obstacles, assigning higher costs to risky zones, and updating its map dynamically, the robot can identify a minimum-cost feasible route based on the information discovered.

This approach demonstrates the application of intelligent agents, rational decision-making, environment characterization, and cost-based AI search to an autonomous rescue problem.